Assignment 4.3

Ryland Jones

2026-04-08

Busiest Airports Analysis

Table 1 shows passenger traffic for Hartsfield-Jackson Atlanta (ATL), Tokyo Haneda (HND), Frankfurt (FRA), Charlotte Douglas (CLT), Beijing Daxing (PKX), and Toronto Pearson (YYZ) airports across the years 2019 through 2022. Atlanta consistently has the highest amount of passengers through all 4 years by a significant amount. Tokyo and Frankfurt also have a good amount of passenger traffic, while Charlotte falls not too far behind. Beijing only appears in the last two years of the data, but, like Toronto, show on the lower numbers of traffic. Overall, the table shows the difference in numbers between international airports from various countries over the four year period.

Table 1: Number of Passengers at Airports Over 4 Years

Airport	IATA	2019	2020	2021	2022
Hartsfield–Jackson Atlanta International Airport	ATL	75704760	93699630	104653451	108067766
Tokyo Haneda Airport	HND	25876429	50290705	78719302	85900167
Frankfurt Airport	FRA	24812849	48918482	59355389	61564957
Charlotte Douglas International Airport	CLT	43302230	47758605	53445770	58811725
Beijing Daxing International Airport	PKX	25051012	NA	NA	49441029
Toronto Pearson International Airport	YYZ	NA	36356109	44761805	46800000

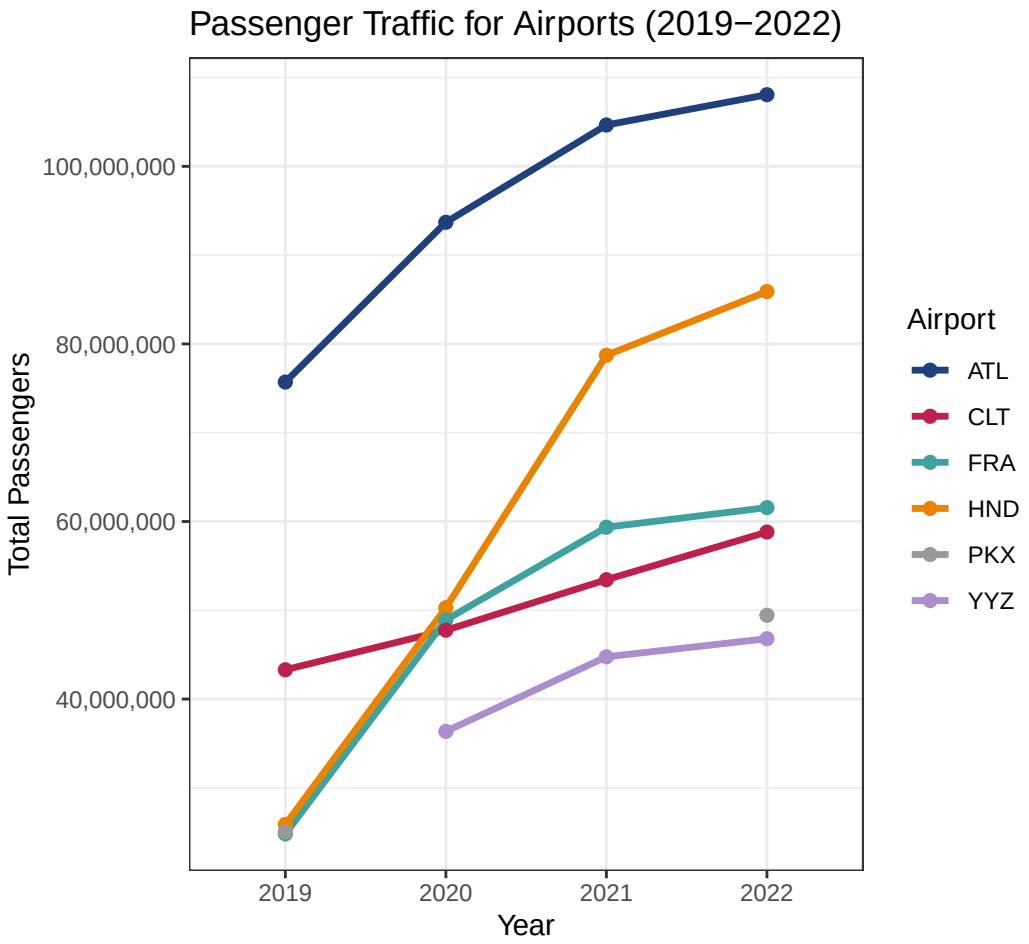
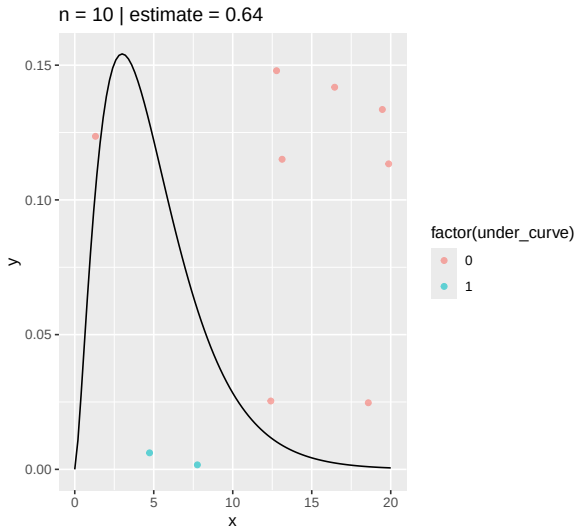


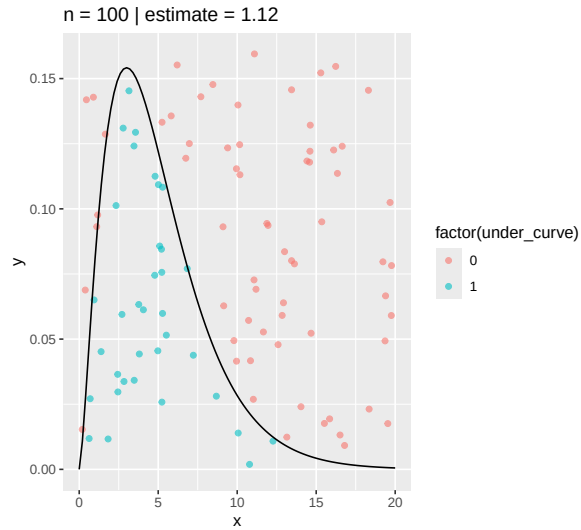
Figure 1: Trend of Passengers at Airports Over 4 Years

Figure 1 visualizes the passenger traffic trends that we see in Table 1 from 2019 to 2022. The line plot makes it easier to compare the changes in traffic over time and the difference between airports. Atlanta and Charlotte showed some decline after a steady increase the first couple years. Toronto and Frankfurt saw a steady increase in traffic through the years, while Tokyo is also seeing an increase after a huge rise from 2019 to 2020. Beijing saw an increase from 2021 to 2022.

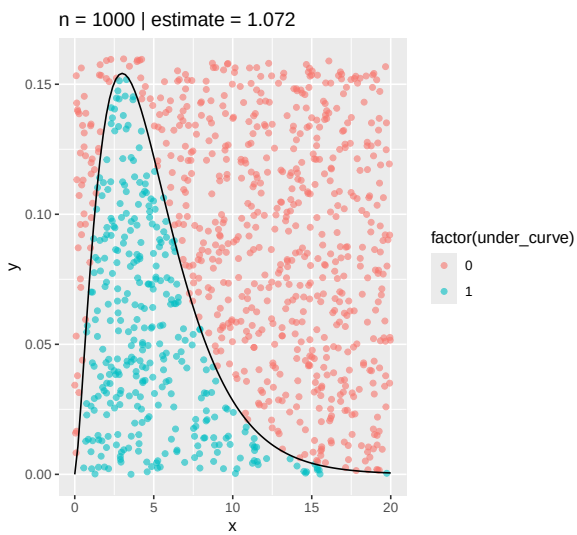
Monte Carlo Simulation



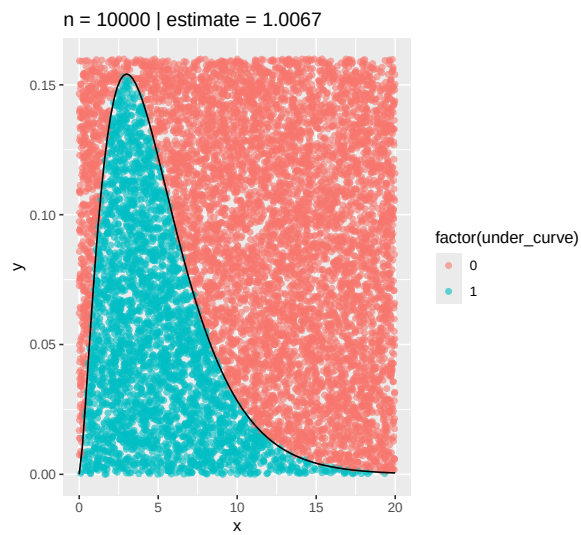
(a) 10 Simulated Points (n = 10)



(b) 100 Simulated Points (n = 100)



(c) 1000 Simulated Points (n = 1000)



(d) 10000 Simulated Points (n = 10000)

Figure 2: Monte Carlo Simulation (Chi-squared)

The Monte Carlo Simulation estimates the area under the curve. The goal is to randomly generate points inside a rectangle that covers the function given, and check how many points fall under the curve. The proportion of points under the curve is multiplied by the area of the rectangle to estimate the integral. Figure 2 demonstrates a chi-squared distribution with 5

degrees of freedom within the region 0 <= x <= 20 and 0 <= y <= 0.16. Random points were generated in this box and each point was checked to see if it fell under the curve of the function. As the sample size increases (n = 10, 100, 1000, 10000), the estimate gets closer to the real value since the points start filling the rectangle more. The exact value of the integral should be 1, as a chi-squared distribution is a probability density function, in which the total area under is always 1. The higher the number of points goes, the close the estimate gets to 1. The multiple plots in Figure 2 help show this. With the small n, the points are everywhere, but as we up the number, the points start to match the curve better.

Calcium

Plan

Goals:

Create a plan to tidy the calcium data and create a plot comparing each woman

Needs:

Nouns: calcium.csv, R, tidyverse

Verbs: Make, create, get, split, call, give, combine, drop, assign, ensure

Steps:

1)Make a plan

2)Get calcium.csv

3)Create new R script

4)Get tidyverse package

5)Split rows 1-4 from rows 5-8

6)Call rows 1-4 the null group and give them the tag null

7)Call rows 5-8 the treatment group and give them the tag diet_exercise

8)Combine rows

9)Drop any NAs

10)Assign ID number to each row

11)Pivot the data to a longer format

12)Create the plot using the x as the year and y as the calcium level

13)Ensure each woman is a line on the plot

Claude's Result From Plan

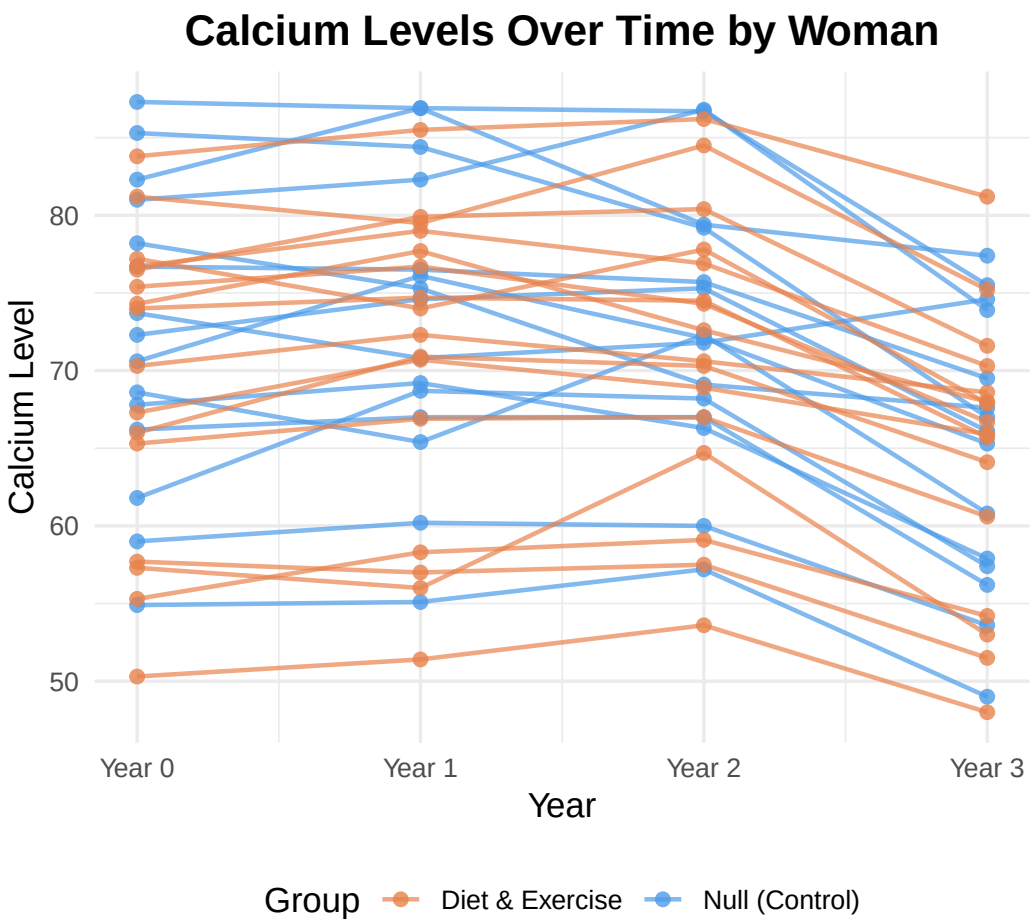


Figure 3: Trend of Women's Calcium Levels Over 3 Years After Taking Calcium

Between the result I got when I gave my plan to Claude (Figure 3) and the generic result (Figure 4), the plots are overall pretty similar. Both show the same data on relatively similar axes, with the generic giving more in-depth labels for years and having a larger range for the calcium level. The plan-given plot gives labels for the colors, while the generic plot gives generic ones. The generic prompt, however, gives averages for each year for both groups, while the prompt with my plan did not. Overall, they are both solid plots that show the data that it was asked to.

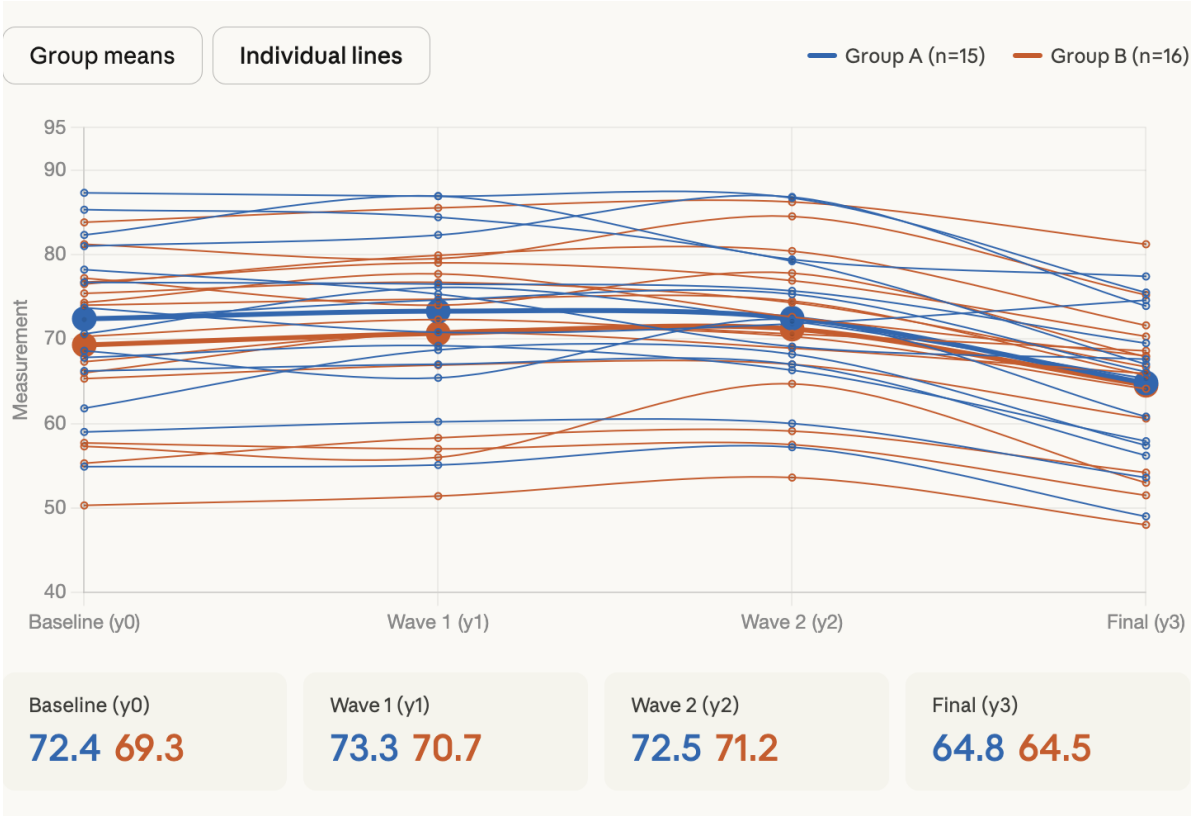


Figure 4: Generic Prompt

Generic Result

Self-Reflection

Going into this class, I was excited to learn a new programming language. Overall, I feel I have gotten a lot out of this class and have been able to learn a lot, whether it be about planning ahead, tidying data, web scraping, quarto, or github. I especially enjoyed learning about web scraping, as I thought it was cool and opens up a lot of doors with what you can do, as long as it is done ethically. There are parts where I struggled, but I feel I was able to pick it up enough to where I wasn't behind. Compared to when I first started this class, I feel my skills have developed quite a bit in this language. However, there is still a lot of room for improvement, especially with tidying data. I am looking forward to continue to learn new things in regard to R.

GenAI Usage Appendix

Used GenAI for the section with Claude and the calcium data

Version: Claude Sonnet 4.6 Extended

(This chat took place on Wednesday, April 15th, 2026.)

Prompt:

Calcium.csv

Goals:

Create a plan to tidy the calcium data and create a plot comparing each woman

Needs:

Nouns: calcium.csv, R, tidyverse

Verbs: Make, create, get, split, call, give, combine, drop, assign, ensure

Steps:

1)Make a plan

2)Get calcium.csv

3)Create new R script

4)Get tidyverse package

5)Split rows 1-4 from rows 5-8

6)Call rows 1-4 the null group and give them the tag null

7)Call rows 5-8 the treatment group and give them the tag diet_exercise

8)Combine rows

9)Drop any NAs

10)Assign ID number to each row

11)Pivot the data to a longer format

12)Create the plot using the x as the year and y as the calcium level

13)Ensure each woman is a line on the plot

Response:

Here's the plan and what the script does, step by step:

Data structure discovered: The CSV has 8 columns — the left 4 (y0–y3) belong to the null/control group and the right 4 (y0–y3) belong to the diet & exercise group. Each of the 16 rows is a different woman.

Script walkthrough:

Read — skips the raw header and assigns clean column names (null_y0…trt_y3).

Split — selects columns 1–4 → null group; columns 5–8 → diet_exercise group; adds a group tag to each.

Combine — bind_rows() stacks both groups.

Drop NAs — drop_na() removes the one incomplete row (woman 16 in the treatment side had all NAs).

Assign IDs — row_number() gives each woman a unique woman_id. Pivot longer — reshapes y0, y1, y2, y3 into a single year + calcium column pair.

Plot — ggplot2 draws one line per woman_id, grouped and coloured by group, with year on the x-axis and calcium level on the y-axis.

Run it in R from the same directory as calcium.csv and it will display the plot and save it as calcium_plot.png.

Code Appendix

```r
# Make a table of busiest airports ----
library(tidyverse)
library(rvest)
library(knitr)
library(kableExtra)

airportRaw <- read_html(
  x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |>
  html_elements(css = "table") |>
  html_table()

airportData <- bind_rows(
  mutate(airportRaw[[2]], Year = 2022),
  mutate(airportRaw[[3]], Year = 2021),
  mutate(airportRaw[[4]], Year = 2020),
  mutate(airportRaw[[5]], Year = 2019)
)

airportData <- airportData |>
  separate_wider_delim(`Code(IATA/ICAO)`, delim = "/", names = c("IATA", "ICAO")) |>
  filter(IATA %in% c("ATL", "FRA", "PKX", "HND", "CLT", "YYZ")) |>
  select(Year, Airport, IATA, Totalpassengers) |>
  mutate(
    Totalpassengers = as.numeric(
      gsub(",", "", gsub("\\[.*?\\]", "", Totalpassengers))
    )
  ) |>
  pivot_wider(
    names_from = Year,
    values_from = Totalpassengers
  )

airportData |>
  select(Airport, IATA, `2019`, `2020`, `2021`, `2022`) |>
  kable(
    digits = 0,
    format = "latex",
  ) |>
  kable_styling(
    latex_options = "scale_down",
    font_size = 12
```

```r
  )

# Set Color Palette ----
psuPalette <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300",
                "#999999", "#AC8DCE", "#F2665E", "#99CC00")
# Make a plot of busiest airports ----
library(scales)

airport_long <- airportData |>
  pivot_longer(
    cols = c(`2019`, `2020`, `2021`, `2022`),
    names_to = "Year",
    values_to = "Passengers"
  )

ggplot( #create plot----
  data = airport_long,
  mapping = aes(
    x = Year,
    y = Passengers,
    color = IATA,
    group = IATA
  )
) +
  geom_line(size = 1.2) + #lines----
  geom_point(size = 2) + #dots to emphasize years----
  labs( #Create labels----
    title = "Passenger Traffic for Airports (2019-2022)",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  scale_y_continuous(
    labels = comma
    ) +
  scale_color_manual( #Assign color palette----
    values = psuPalette
  ) +
  theme_bw()

# Make a plot of the monte carlo simulation ----
library(tidyverse)
```

```r
generate_points <- function(n, x_min, x_max, y_min, y_max){ #Point generating function----
  data.frame(
    x = runif(n, x_min, x_max),
    y = runif(n, y_min, y_max)
  )
}

#x and y values----
x_min <- 0
x_max <- 20
y_min <- 0
y_max <- 0.16

#chi-squared function----
f <- function(x){
  dchisq(x, df = 5)
}

#Plot function with adjustable n----
make_plot <- function(n){

  points <- generate_points(n, x_min, x_max, y_min, y_max) |>
    mutate(
      y_curve = f(x),
      under_curve = ifelse(y <= y_curve, 1, 0)
    )

  rectangle_area <- (x_max - x_min) * (y_max - y_min) #Calculate rectangle area----
  estimate <- mean(points$under_curve) * rectangle_area #Calculate estimate----

  ggplot(data = points, #Create plot----
         aes(x = x,
             y = y,
             color = factor(under_curve)
             )) +
    geom_point(alpha = 0.6) +
    stat_function(fun = dchisq,
                  args = list(df = 5),
                  xlim = c(0,20),
                  color = "black") +
    ggtitle(paste("n =", n, "| estimate =", round(estimate, 4)))
}
```

```r
make_plot(10)
make_plot(100)
make_plot(1000)
make_plot(10000)

# Calcium Data Analysis
# Tidying data and plotting calcium levels per woman over time

library(tidyverse)

# ------------------------------------------------------------------------------
# Step 1: Read raw data
# ------------------------------------------------------------------------------
raw <- read_csv("C:/Users/rythe/Downloads/calcium.csv", col_names = FALSE)

# The file has 8 columns: y0,y1,y2,y3 (null group) | y0,y1,y2,y3 (treatment group)
# Rows 1-4  (rows 2-5 in raw, skipping header) → null group
# Rows 5-8  (rows 6-9 in raw)                  → diet_exercise group
# But actually the CSV has a header row, so after read_csv with col_names=FALSE:
#   row 1 = header text, rows 2-17 = data (16 women, 8 per group side)

# Re-read with proper column names
raw <- read_csv(
  "C:/Users/rythe/Downloads/calcium.csv",
  col_names = c("null_y0", "null_y1", "null_y2", "null_y3",
                "trt_y0",  "trt_y1",  "trt_y2",  "trt_y3"),
  skip = 1   # skip the original header row
)

# ------------------------------------------------------------------------------
# Step 2: Split into two groups
# ------------------------------------------------------------------------------

# NULL group (columns 1-4)
null_group <- raw %>%
  select(y0 = null_y0, y1 = null_y1, y2 = null_y2, y3 = null_y3) %>%
  mutate(group = "null")

# DIET_EXERCISE group (columns 5-8)
trt_group <- raw %>%
  select(y0 = trt_y0, y1 = trt_y1, y2 = trt_y2, y3 = trt_y3) %>%
  mutate(group = "diet_exercise")
```

```r
# ------------------------------------------------------------------------------
# Step 3: Combine, drop NAs, assign ID, pivot longer
# ------------------------------------------------------------------------------
combined <- bind_rows(null_group, trt_group) %>%
  drop_na() %>%                                       # drop rows with any NA
  mutate(woman_id = row_number()) %>%                 # unique ID per woman
  pivot_longer(
    cols      = c(y0, y1, y2, y3),
    names_to  = "year",
    values_to = "calcium"
  ) %>%
  mutate(year = as.integer(str_remove(year, "y")))  # convert "y0"→0, etc.

# ------------------------------------------------------------------------------
# Step 4: Plot - one line per woman, coloured by group
# ------------------------------------------------------------------------------
p <- ggplot(combined,
            aes(x     = year,
                y     = calcium,
                group = woman_id,
                colour = group)) +
  geom_line(alpha = 0.7, linewidth = 0.8) +
  geom_point(size = 2, alpha = 0.8) +
  scale_x_continuous(breaks = 0:3, labels = paste0("Year ", 0:3)) +
  scale_colour_manual(
    values = c("null" = "#4C9BE8", "diet_exercise" = "#E8824C"),
    labels = c("null" = "Null (Control)", "diet_exercise" = "Diet & Exercise")
  ) +
  labs(
    title   = "Calcium Levels Over Time by Woman",
    x       = "Year",
    y       = "Calcium Level",
    colour  = "Group"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom"
  )

print(p)
```